Lecture 2 : Model Driven Architecture (MDA)

Ingénierie Dirigée par les Modèles (IDM / MDE) – version 2027

Du modèle métier au code : CIM, PIM, PSM et transformations

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles – Smart Systems Lab
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

- **Pourquoi MDA ?** : *Comment ne plus réécrire le métier à chaque nouvelle technologie ?*
- **Les modèles CIM / PIM / PSM** : *Que contient chaque modèle, sur l'exemple de la bibliothèque ?*
- **Les transformations** : *Comment passe-t-on d'un modèle à l'autre ?*
- **Approches de transformation** : *Avec quels outils écrit-on une transformation ?*
- **Exercices et synthèse** : *Savez-vous classer, projeter et transformer ?*

Objectifs de la séance

À la fin de cette séance, vous serez capables de :

Expliquer

l'idée de MDA : séparer le **métier** de la **technique** (la plateforme).

Distinguer

les modèles **CIM**, **PIM**, **PDM** et **PSM** et dire ce que chacun contient.

Classer

une transformation : PIM→PIM, PIM→PSM, PSM→PIM... endogène / exogène, M2M / M2T.

Écrire

une transformation simple **par template** (XSLT), **par programmation** (Java) ou **par modélisation** (ATL).

! la **bibliothèque** (Adhérent, Livre, Emprunt) : nous la suivrons des besoins (CIM) jusqu'au code Java et SQL.

Rappel de la Lecture 1 et code couleur

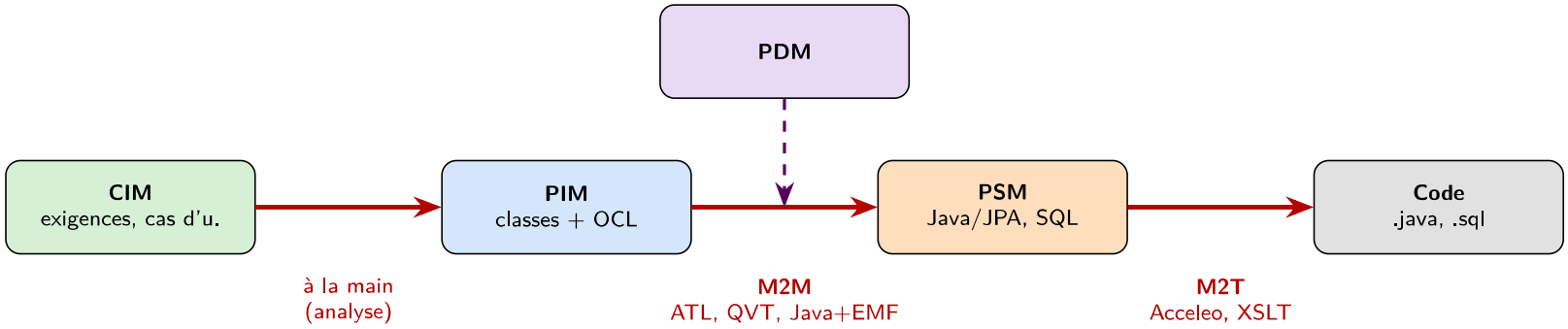
Ce que l'on sait déjà

- Un **modèle** simplifie un système pour un objectif.
- Un modèle (M1) est **conforme** à un méta-modèle (M2), lui-même conforme au MOF (M3).
- L'IDM rend le modèle **productif** : on le vérifie, on le **transforme**, on génère du code.

Ce que l'on va voir

- **MDA** = la démarche IDM proposée par l'**OMG** (2001).
- Plusieurs modèles **du même système**, du plus abstrait au plus technique.
- Les **transformations** qui relient ces modèles.

Le même code couleur dans tout le cours



Pourquoi MDA ?

Comment ne plus réécrire le métier à chaque nouvelle technologie ?

Pourquoi MDA ?

- Complexité **croissante** des logiciels.
- Séparation des **préoccupations** (métier, sécurité, persistance, IHM...).
- Multiplicité des besoins (front office / back office).
- Multiplicité des **plateformes** : Java EE, .NET, PHP + SQL, mobile, cloud...
- Évolution **sans fin** des technologies.

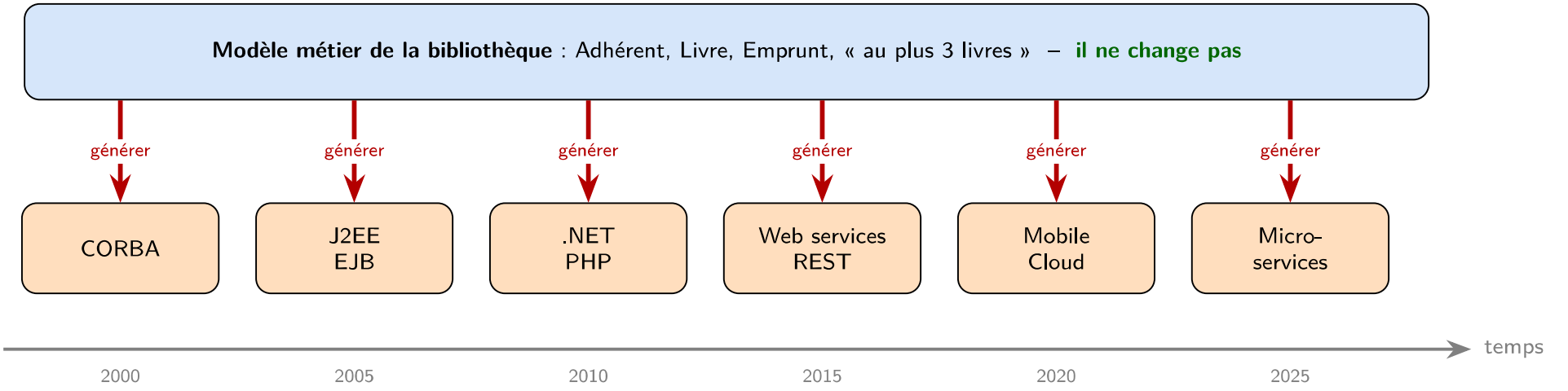
La question

Faut-il continuer à baser les développements sur le **code** des applications ?

Le **métier** d'une bibliothèque (prêter des livres à des adhérents) n'a pas changé depuis 30 ans... mais la technologie a changé **six fois**.

Pourquoi MDA ?

→ Le problème : le métier est stable, la plateforme change



Sans modèle : à chaque nouvelle plateforme on **réécrit** le métier dans le code (coût, erreurs, perte de connaissance).

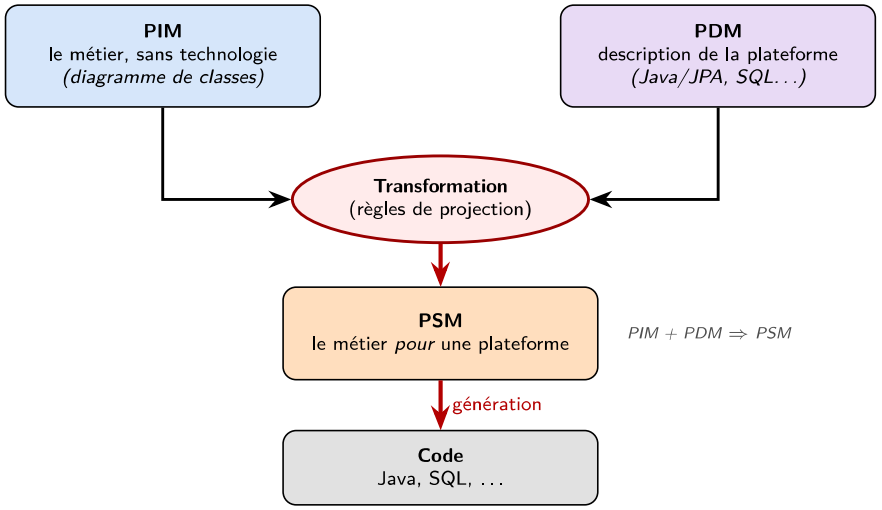
! écrire le métier **une seule fois** dans un modèle, puis **générer** la version de chaque plateforme.

Pourquoi MDA ?

→ Le principe de MDA (Model Driven Architecture)

- Démarche proposée par l'**OMG** en 2001.
- **Séparer** les spécifications **fonctionnelles** (le métier) des spécifications d'**implantation** sur une plateforme.
- Construire d'abord des modèles d'**analyse**, puis de **conception**, jusqu'au **code**...
- ... par **transformations** et **enrichissements successifs** (raffinement).
- Standards utilisés : UML, OCL, MOF, XMI, QVT.

PIM + PDM ⇒ PSM ⇒ Code



Pourquoi MDA ?

→ Une analogie : la recette de cuisine

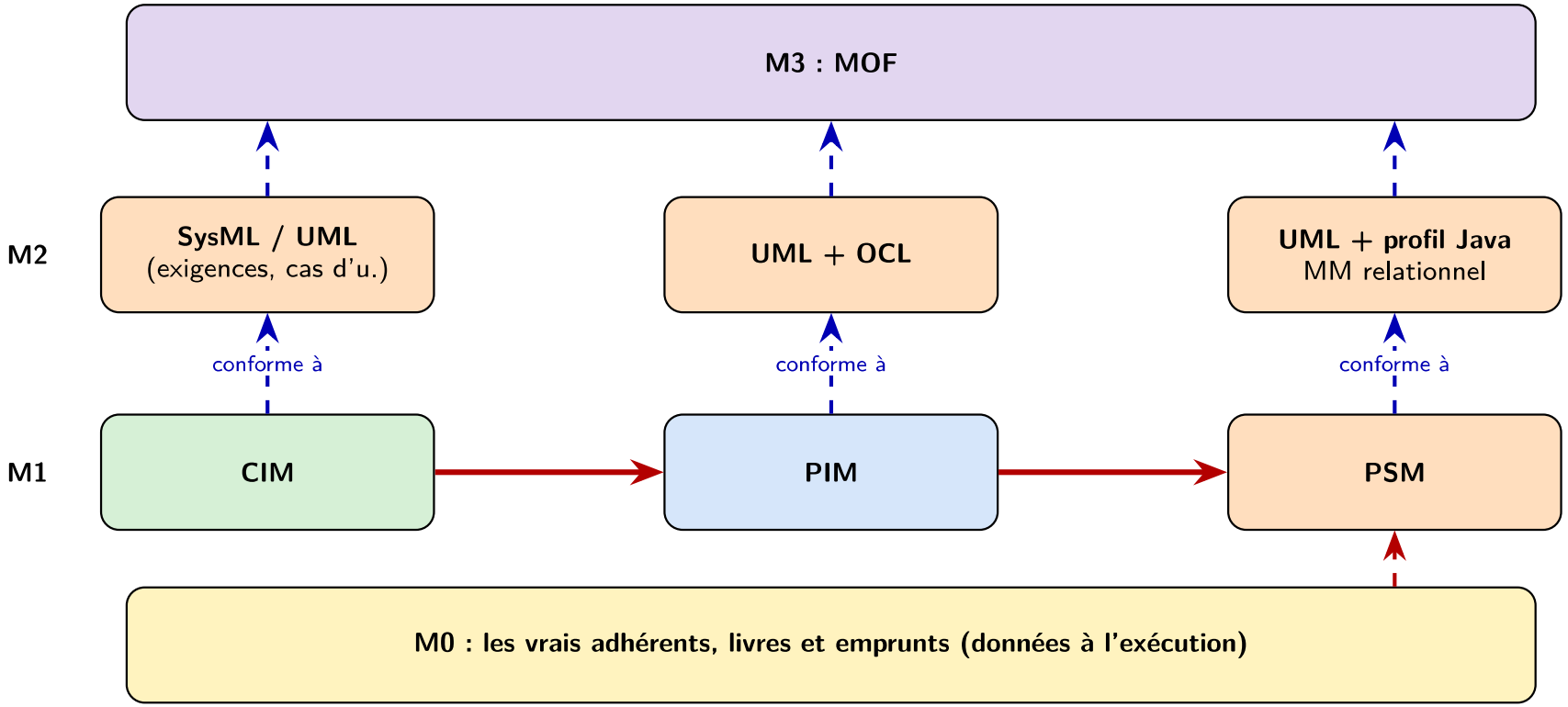
MDA	Cuisine	Bibliothèque
CIM : le besoin	« Je veux un gâteau pour 8 personnes, sans gluten. »	« Les adhérents empruntent au plus 3 livres. »
PIM : la solution sans technologie	La recette : ingrédients, étapes, temps de cuisson.	Diagramme de classes Adherent, Livre + contrainte OCL.
PDM : la plateforme	Le four : électrique, à gaz, micro-ondes...	Java/JPA, SQL, .NET...
PSM : la solution adaptée	La recette adaptée au four à gaz (thermostat 6).	Classes Java @Entity, tables SQL.
Code	Le gâteau !	Adherent.java, bibliotheque.sql



la recette (PIM) ne change pas quand on change de four (plateforme) : seule l'**adaptation** (PSM) change.

Pourquoi MDA ?

→ MDA et les niveaux M0 à M3



! CIM, PIM et PSM sont **tous des modèles M1** du même système. Ils diffèrent par leur **point de vue** (plus ou moins proche de la technique), pas par leur niveau méta.

Question flash

?

Le PIM de la bibliothèque est-il un modèle M1 ou un méta-modèle M2 ? Et le langage UML ?

Question flash

? Le PIM de la bibliothèque est-il un modèle M1 ou un méta-modèle M2 ? Et le langage UML ?

Réponse

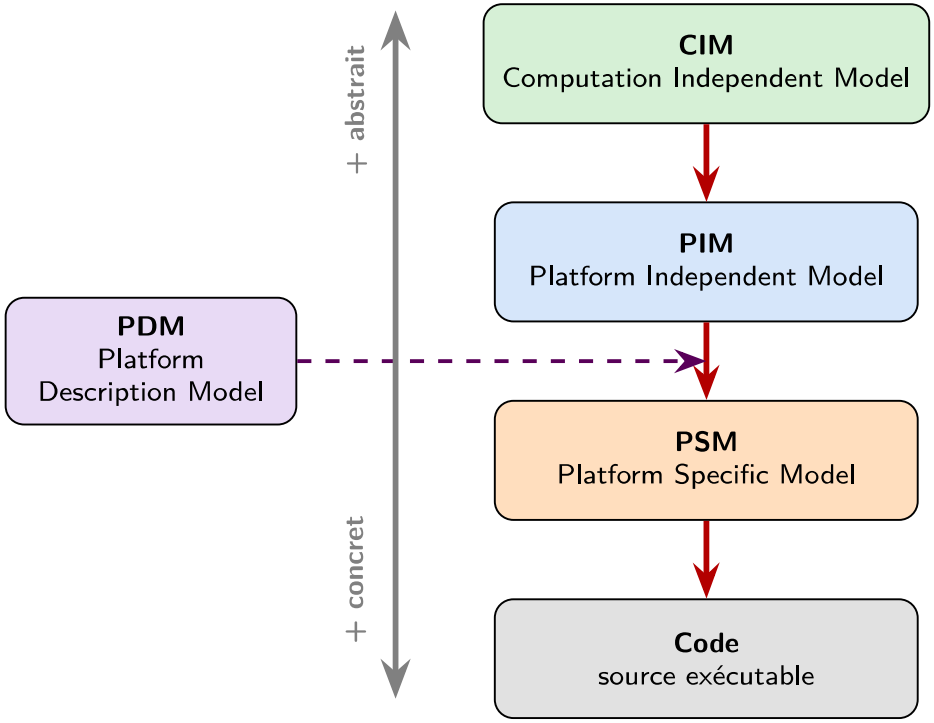
Le PIM est un **modèle (M1)** : il décrit **un** système (la bibliothèque). UML est un **méta-modèle (M2)** : il définit le langage dans lequel le PIM est écrit.

Les modèles CIM / PIM / PSM

Que contient chaque modèle, sur l'exemple de la bibliothèque ?

Les modèles de MDA

→ Vue d'ensemble : du plus abstrait au plus concret



QUOI ? les besoins, le vocabulaire du métier.
Vue de l'utilisateur.

COMMENT ? la solution logicielle, **sans** technologie.
Vue de l'analyste / concepteur.

AVEC QUOI ? la solution sur **une** plateforme (Java, SQL...).
Vue de l'architecte.

EXÉCUTER : le code généré (complété à la main si besoin).
Vue du développeur.

Chaque modèle répond à une **question** différente et s'adresse à un **public** différent.

Le besoin exprimé par le client

« Notre bibliothèque prête des livres à ses **adhérents**. Un adhérent peut **rechercher** un livre, l'**emprunter** et le **rendre**. Il emprunte **au plus 3 livres** en même temps, pour **15 jours** au maximum. Un livre emprunté n'est plus disponible. Le **bibliothécaire** gère le catalogue. »

> **!** repérer les **noms** (futurs concepts : Adhérent, Livre, Emprunt) et les **verbes** (futures fonctions : emprunter, rendre...).

Ce que nous allons construire

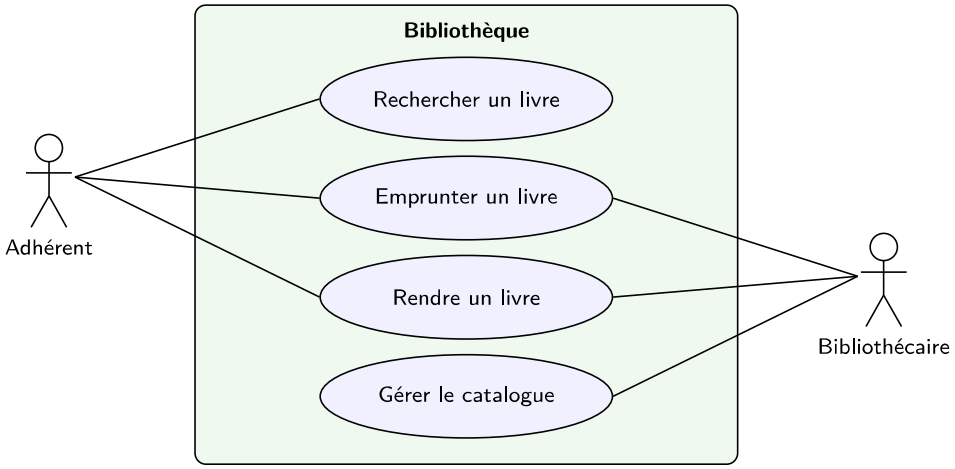
- **CIM** : cas d'utilisation + exigences.
- **PIM** : diagramme de classes + OCL.
- **PDM** : règles Java/JPA et SQL.
- **PSM** : classes Java annotées, tables.
- **Code** : Adherent.java, bibliotheque.sql.

CIM : Computation Independent Model

→ Le modèle des exigences (du métier)

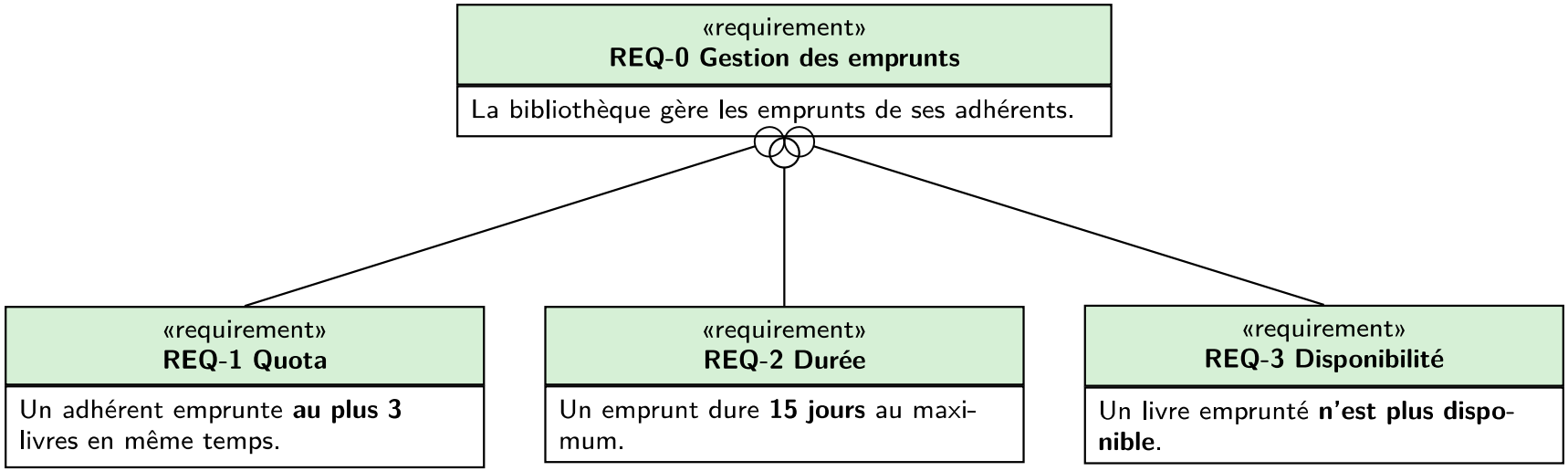
- Modèle des **exigences**, du **métier** ou du **domaine**.
- Décrit **ce que** le système doit faire : les acteurs, les flux, les actions.
- **Aucune** notion informatique : on peut le montrer au client.
- Diagrammes : **cas d'utilisation**, **exigences (SysML)**, packages, activités métier.

! le client, l'expert métier, l'analyste.



CIM : Computation Independent Model

→ Exemple : le diagramme d'exigences (SysML)

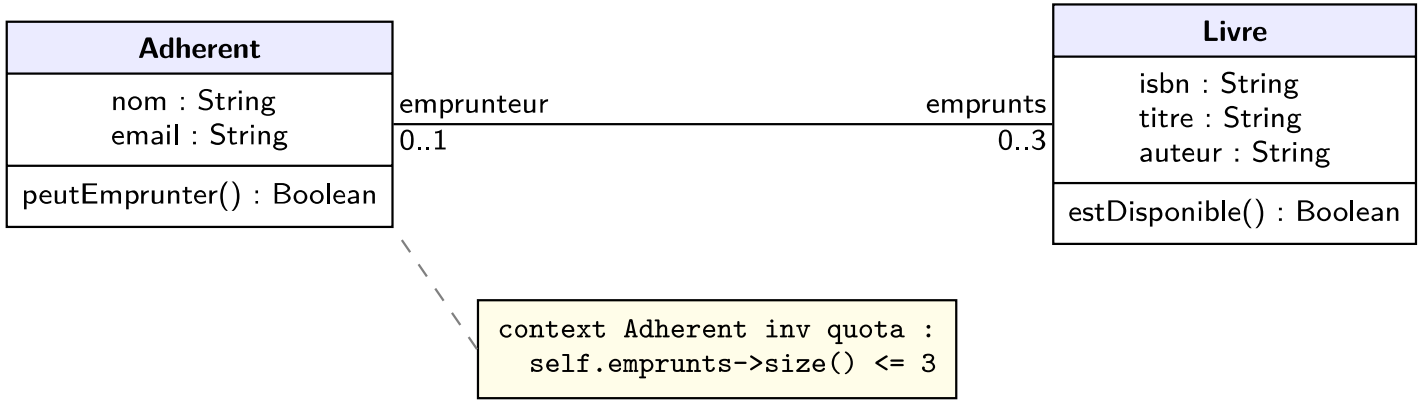


(diagramme d'exigences SysML : aucune notion informatique)

Chaque exigence est **numérotée** : on pourra vérifier plus tard qu'elle est **prise en compte** par le PIM, le PSM et le code (**traçabilité**).

PIM : Platform Independent Model

- Décrit la **solution logicielle** : la structure (classes) et le comportement (états, séquences, activités).
- **Indépendant** de toute plateforme : pas de Java, pas de SQL, pas d'annotation.
- Types **abstraits** : String, Integer, Boolean, Date.
- Les règles métier deviennent des **contraintes OCL** (REQ-1 ⇒ « au plus 3 »).
- Diagrammes : classes (+ OCL), objets, composants, états-transitions, séquences, activités.



PIM : Platform Independent Model

→ Ce que le PIM contient... et ne contient pas

Le PIM contient

- Les **concepts** du métier : Adherent, Livre.
- Leurs **attributs** avec des types abstraits.
- Les **associations** et multiplicités (0..1, 0..3).
- Les **opérations** métier : peutEmprunter(), estDisponible().
- Les **contraintes** (OCL).

Le PIM ne contient pas

- D'**identifiant technique** (id auto-incrémenté).
- De type de plateforme : VARCHAR, List<...>, LocalDate.
- D'**annotation** : @Entity, @Id...
- De détail d'**IHM**, de framework, de base de données.

Test simple

si le mot « Java », « SQL » ou « table » apparaît : ce n'est plus un PIM !

PDM : Platform Description Model

→ Décrire la plateforme cible

- Décrit la **structure** et les **services techniques** d'une plateforme d'exécution (Java EE, .NET, SQL, CORBA...).
- Contient les **règles de passage** du PIM vers cette plateforme (« une classe devient... »).
- En pratique : un **profil UML**, un **méta-modèle** de la plateforme et des **règles** écrites en QVT, ATL...

Concept du PIM (UML)	Plateforme Java / JPA	Plateforme SQL
Classe `C`	classe `C` annotée `@Entity`	table `C`
Identité d'un objet (implicite)	attribut `@Id Long id`	colonne `id PRIMARY KEY`
Attribut `String`	`private String` + get/set	colonne `VARCHAR(255)`
Attribut `Integer` / `Boolean`	`int` / `boolean`	`INTEGER` / `BOOLEAN`
Attribut `Date`	`java.time.LocalDate`	`DATE`
Association `0..1` – `0..*`	`List<B>` `@OneToMany` / `A` `@ManyToOne`	clé étrangère côté « plusieurs »
Association `*` – `*`	`List` des deux côtés `@ManyToMany`	table de jointure
Opération	méthode `public`	(pas de traduction)

PSM : Platform Specific Model

→ Le PSM Java / JPA

- Modèle **technique** : le PIM **projeté** sur une plateforme.
- Décrit l'implémentation sur une plateforme **particulière**.
- Utilise des **profils UML** : UML Profile for Java/EJB, for CORBA, for .NET, for SQL...

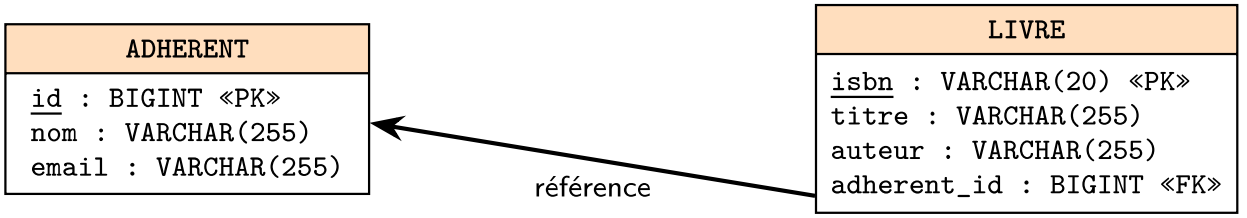
Ce qui a changé par rapport au PIM

- Ajout d'un **id** technique.
- Types **Java** (List<Livre>).
- Stéréotypes **«Entity»**, **«Id»**, **«OneToMany»**.



PSM : Platform Specific Model

→ Le PSM relationnel (SQL)



- Chaque **classe** du PIM est devenue une **table** ; chaque **attribut** une **colonne**.
- L'association 0..1 — 0..3 est devenue une **clé étrangère** adherent_id dans LIVRE (côté « plusieurs »).
- La contrainte « au plus 3 » n'a **pas** d'équivalent direct en SQL : elle sera vérifiée par le code (ou un trigger).

Code source

→ Le résultat final du processus


```java
@Entity
public class Adherent {
    @Id @GeneratedValue
    private Long id;
    private String nom;
    private String email;

    @OneToMany(mappedBy = "emprunteur")
    private List<Livre> emprunts = new ArrayList<>();

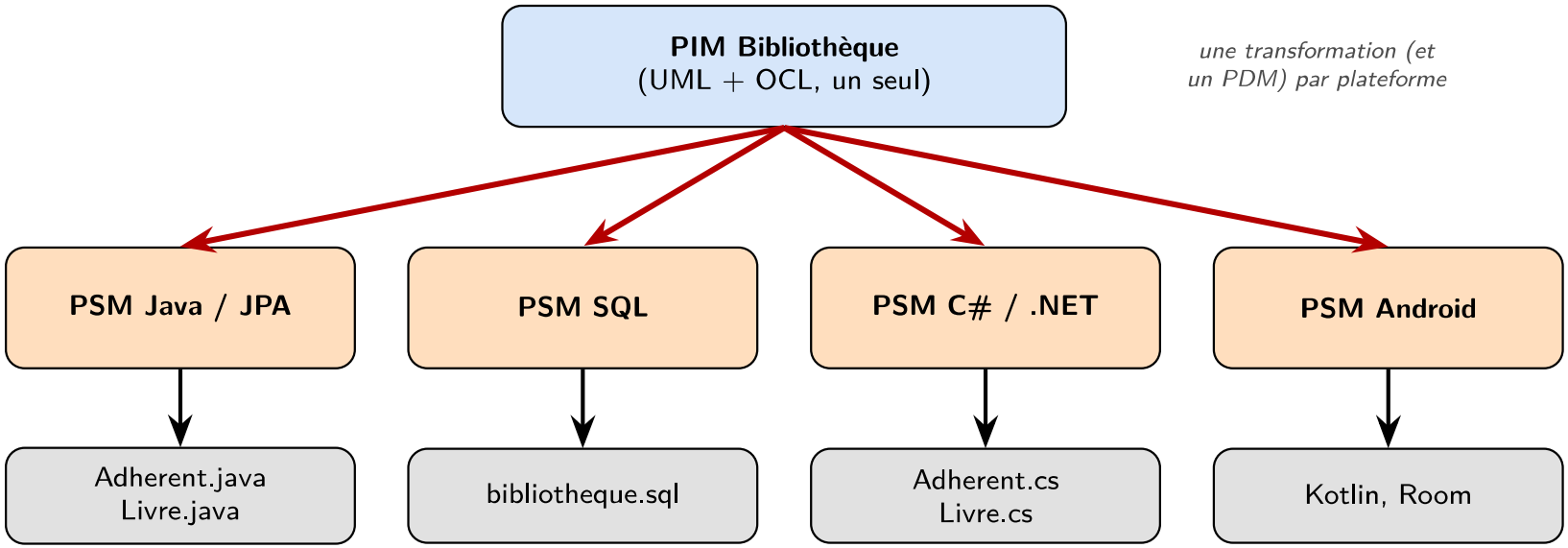
    public boolean peutEmprunter() {   // REQ-1 / contrainte OCL
        return emprunts.size() < 3;
    }
    // + getters / setters générés
}
```

```sql
CREATE TABLE ADHERENT (
  id     BIGINT PRIMARY KEY,
  nom    VARCHAR(255),
  email  VARCHAR(255)
);
CREATE TABLE LIVRE (
  isbn         VARCHAR(20) PRIMARY KEY,
  titre        VARCHAR(255),
  auteur       VARCHAR(255),
  adherent_id BIGINT REFERENCES ADHERENT(id)
);
```

- Obtenu par **génération automatique** (totale ou partielle) à partir du PSM.
- Peut être **complété** à la main (corps des méthodes métier).

Les modèles de MDA

→ Un seul PIM, plusieurs PSM



pour une nouvelle plateforme, on **réutilise** le PIM et on écrit (ou on réutilise) une **nouvelle transformation**. Le métier n'est pas réécrit.

Les modèles de MDA

→ Synthèse

Modèle	Question	Contenu	Bibliothèque	Notations
CIM	Quoi ?	besoins, acteurs, règles métier	« au plus 3 livres »	cas d'utilisation, SysML
PIM	Comment, sans techno ?	classes, comportement, contraintes	Adherent, Livre, OCL	UML + OCL
PDM	Sur quelle plateforme ?	concepts et règles de la plateforme	Java/JPA, SQL	profil UML, méta-modèle
PSM	Comment, avec la techno ?	PIM projeté sur la plateforme	@Entity, tables	UML + profil
Code	Exécuter	source généré + complété	.java, .sql	langage cible

Question flash

?

Classer en CIM, PIM, PSM ou Code : (a) « un adhérent emprunte au plus 3 livres » ; (b) la classe Livre avec isbn : String ; (c) la table LIVRE avec adherent_id BIGINT ; (d) le fichier Livre.java ; (e) la classe Livre «Entity» avec «Id» isbn ; (f) le cas d'utilisation « Rendre un livre ».

Question flash

?

Classer en CIM, PIM, PSM ou Code : (a) « un adhérent emprunte au plus 3 livres » ; (b) la classe Livre avec isbn : String ; (c) la table LIVRE avec adherent_id BIGINT ; (d) le fichier Livre.java ; (e) la classe Livre «Entity» avec «Id» isbn ; (f) le cas d'utilisation « Rendre un livre ».

Réponse

(a) **CIM** · (b) **PIM** · (c) **PSM** (relationnel) · (d) **Code** · (e) **PSM** (Java) · (f) **CIM**.

Les transformations

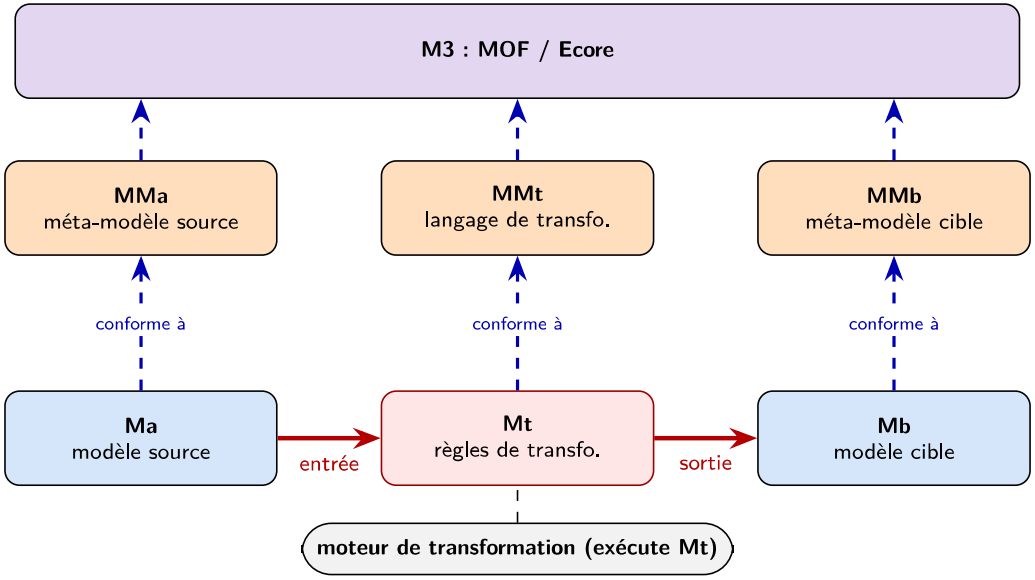
Comment passe-t-on d'un modèle à l'autre ?

Les transformations

→ Qu'est-ce qu'une transformation de modèles ?

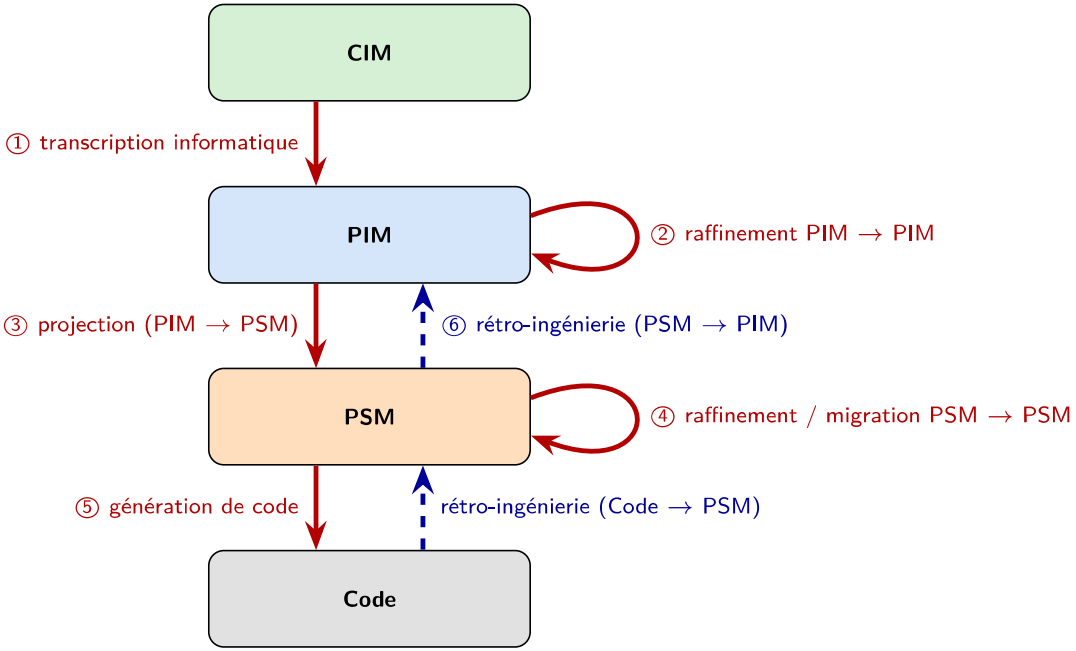
- Une **transformation** produit un modèle **cible** (Mb) à partir d'un modèle **source** (Ma).
- Elle est définie par des **règles** écrites sur les **méta-modèles** (MMa, MMb), pas sur un modèle particulier.
- ⇒ la même transformation sert pour **tous** les modèles conformes à MMa.
- Un **moteur** exécute les règles.

> **!** la règle « toute **Classe** devient une **Table** » s'applique à Adherent, à Livre... et à n'importe quel autre PIM.



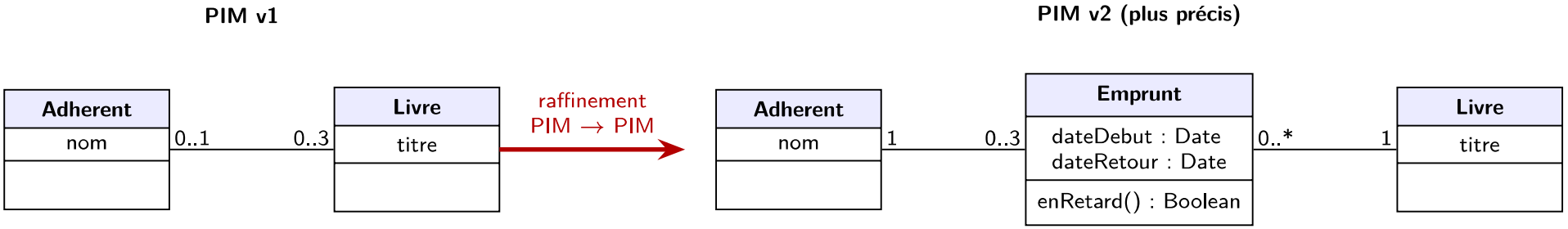
Les transformations

→ Les transformations de MDA



- ① **CIM** → **PIM** : transcription informatique (souvent **à la main**).
- ② **PIM** → **PIM** : raffinement.
- ③ **PIM** → **PSM** : projection vers une technologie.
- ④ **PSM** → **PSM** : raffinement, migration.
- ⑤ **PSM** → **Code** : génération de code.
- ⑥ **PSM** → **PIM** : rétro-ingénierie.

- On **ajoute** de l'information **non liée** à une plateforme pour rendre le modèle plus précis.
- Exemples : créer des classes à partir des diagrammes de séquences ; créer un diagramme d'états à partir d'un diagramme de classes ; **réifier** une association en classe.



*On ajoute une information **métier** (l'historique des emprunts, les dates) : toujours **aucune** technologie.*

- On **ajoute** au PIM les informations propres à **une** plateforme.
- La plateforme visée est décrite par un **PDM**.
- Les règles sont **généralisées** et **capitalisées** dans un langage de transformation : on les réutilise pour d'autres projets.

Règles PIM → Java / JPA (extrait)

- Pour chaque **Classe** C → une classe Java C annotée @Entity.
- Si C n'a pas d'identifiant → ajouter @Id Long id.
- Pour chaque **Attribut** → un champ private + get/set.
- Pour chaque association 0..1 — * → List<...> + @OneToMany.



④ PSM → PSM : raffinement / migration

- Raffinement : PSM Java → PSM Java + Spring (ajout des contrôleurs REST).
- **Migration** : PSM .NET → PSM Java.
- Parfois à travers un formalisme intermédiaire (ex. **SDL**) pour générer le code.

⑥ PSM → PIM : rétro-ingénierie

- On **remonte** d'une plateforme vers le métier (reverse engineering).
- Exemple : lire le schéma SQL d'une vieille base et retrouver le diagramme de classes.
- Indispensable pour **intégrer** ou **moderniser** des applications existantes.

! ancienne base SQL —⑥→ PIM —③→ PSM Java/JPA —⑤→ code neuf : le métier est **récupéré** puis **régénéré** sur une nouvelle plateforme.

Les transformations

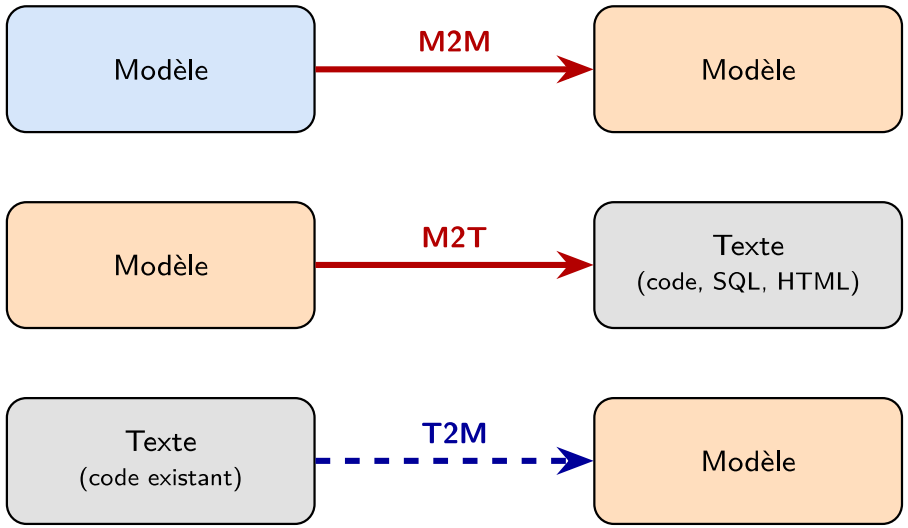
→ Taxonomie : endogène / exogène, horizontale / verticale

	ENDOGÈNE MMa = MMb (même langage)	**EXOGÈNE** MMa ≠ MMb (langages différents)
HORIZONTALE même niveau d'abstraction	**Restructuration** (refactoring) *ex. : renommer, factoriser une super-classe dans le PIM*	**Migration**, fusion *ex. : PSM .NET → PSM Java ; fusionner deux modèles*
VERTICALE le niveau d'abstraction change	**Raffinement** *ex. : PIM v1 → PIM v2 (on ajoute la classe Emprunt)*	**Projection / génération** *PIM → PSM, PSM → Code* et **rétro-ingénierie** (sens inverse)

Endogène : source et cible ont le **même** méta-modèle. **Exogène** : méta-modèles **différents**.
Verticale : le niveau d'abstraction change. **Horizontale** : il reste le même.

Les transformations

→ Selon la nature de la cible : M2M, M2T, T2M



Model to Model : PIM → PSM
outils : ATL, QVT, Java + EMF

Model to Text : PSM → Code
outils : Acceleo, XSLT, Xtend

Text to Model : rétro-ingénierie
outils : Xtext, MoDisco

! PIM → PSM est en général une transformation **M2M** ; PSM → Code est une transformation **M2T** (génération de texte).

Question flash

?

Classer chaque transformation (endogène/exogène, horizontale/verticale) : (a) PIM Bibliothèque → schéma SQL ; (b) renommer « Adherent » en « Membre » dans le PIM ; (c) PSM .NET → PSM Java ; (d) schéma SQL existant → diagramme de classes.

Question flash

?

Classer chaque transformation (endogène/exogène, horizontale/verticale) : (a) PIM Bibliothèque → schéma SQL ; (b) renommer « Adherent » en « Membre » dans le PIM ; (c) PSM .NET → PSM Java ; (d) schéma SQL existant → diagramme de classes.

Réponse

(a) **exogène, verticale** (projection) · (b) **endogène, horizontale** (restructuration) · (c) **exogène, horizontale** (migration) · (d) **exogène, verticale** (rétro-ingénierie).

Approches de transformation

Avec quels outils écrit-on une transformation ?

Approches de transformation

→ Trois grandes approches

Par template

La transformation est un **modèle de texte** à trous, rempli avec les données du modèle source.

Outils : **XSLT**, **Acceleo**, Velocity, Xtend.

Par programmation

La transformation est un **programme** (Java...) qui lit et écrit les modèles via des **API**.

Outils : Java + **EMF**.

Par modélisation

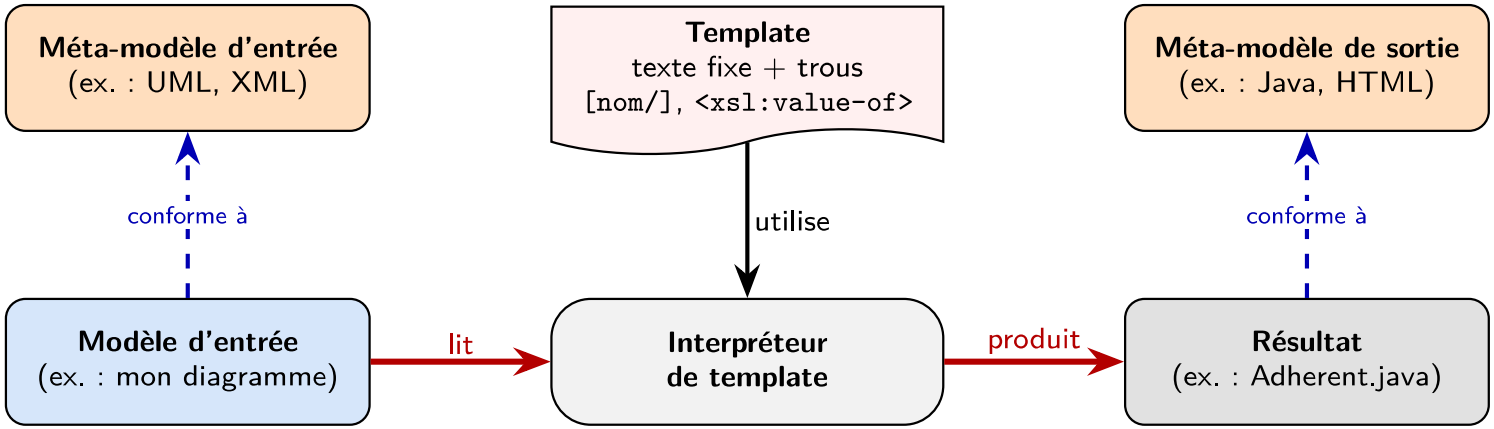
La transformation est elle-même un **modèle**, conforme au méta-modèle d'un langage dédié.

Outils : **QVT**, **ATL**.

! template pour produire du **texte** (code, HTML) ; programmation pour des règles **complexes** ; modélisation pour des transformations **réutilisables** M2M.

Transformation par template

→ Principe



- Le **template** = du texte **fixe** (ce qui ne change pas) + des **trous** (ce qui vient du modèle).
- Un **interpréteur** parcourt le modèle et remplit les trous.
- Idéal pour les transformations **M2T** : génération de code, de HTML, de documentation.

Transformation par template

→ Exemple 1 : XML → texte avec XSLT

Le modèle : article.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="article_txt.xsl
      "?>
<Article>
  <Title>Mon article</Title>
  <Authors>
    <Author>M. Foo</Author>
    <Author>M. Bar</Author>
  </Authors>
</Article>
```

Le texte généré

```
Article - Mon article
Auteurs :
  - M. Foo
  - M. Bar
```

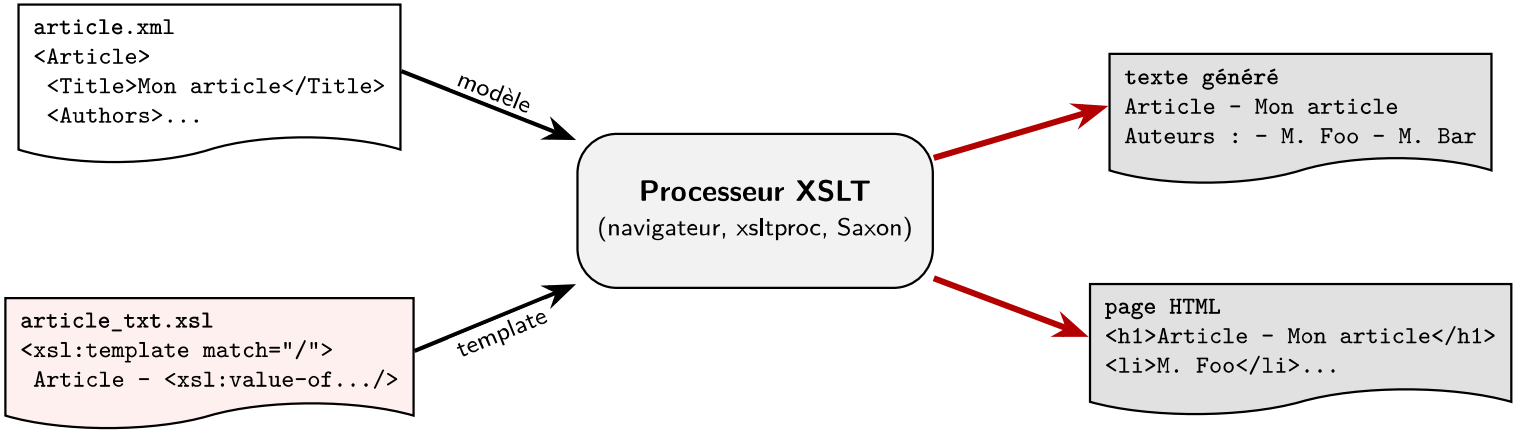
Le template : article_txt.xsl

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/
      XSL/Transform">
  <xsl:output method="text" encoding="UTF-8"/>
  <xsl:template match="/">
    <xsl:text>Article - </xsl:text>
    <xsl:value-of select="/Article/Title"/>
    <xsl:text>&#10;Auteurs :&#10;</xsl:text>
    <xsl:for-each select="/Article/Authors/Author">
      <xsl:text>  - </xsl:text>
      <xsl:value-of select="."/>
      <xsl:text>&#10;</xsl:text>
    </xsl:for-each>
  </xsl:template>
</xsl:stylesheet>
```

Transformation par template

→ Lire un template XSLT

Instruction XSLT	Rôle	Dans l'exemple
xsl :template match="/"	règle appliquée à la racine du document	point de départ
xsl :value-of select="..."	insère une **valeur** du modèle (un trou)	le titre « Mon article »
xsl :for-each select="..."	**boucle** sur des éléments	un tiret par auteur
xsl :output method="text"	type de sortie : text, html, xml	texte brut



Transformation par template

→ Exemple 1 (suite) : le même modèle → HTML

Template article_html.xsl (extrait)

```
<xsl:template match="/">
  <html><body>
    <h1>Article - <xsl:value-of select="/Article/Title"/></h1>
    <ul>
      <xsl:for-each select="/Article/Authors/Author">
        <li style="color:red"><xsl:value-of select="."/></li>
      </xsl:for-each>
    </ul>
  </body></html>
</xsl:template>
```

HTML généré

```
<html><body>
  <h1>Article - Mon article</h1>
  <ul>
    <li style="color:red">M. Foo</li>
    <li style="color:red">M. Bar</li>
  </ul>
</body></html>
```

! **même modèle**, **autre template** ⇒ autre résultat. C'est exactement « un PIM, plusieurs PSM ».

Transformation par template

→ Exemple 2 : PIM → Java avec Acceleo

Template Acceleo

```
[module genJava('http://ensias.ma/pim')]

[template public genClasse(c : Classe)]
[file (c.nom + '.java', false, 'UTF-8')]
public class [c.nom/] {
[for (a : Attribut | c.attributs)]
    private [a.type.javaType()/] [a.nom/];
[/for]
}
[/file]
[/template]
```

[c.nom/] et [a.nom/] sont les **trous** ; [for . . .] est une **boucle** sur les attributs.

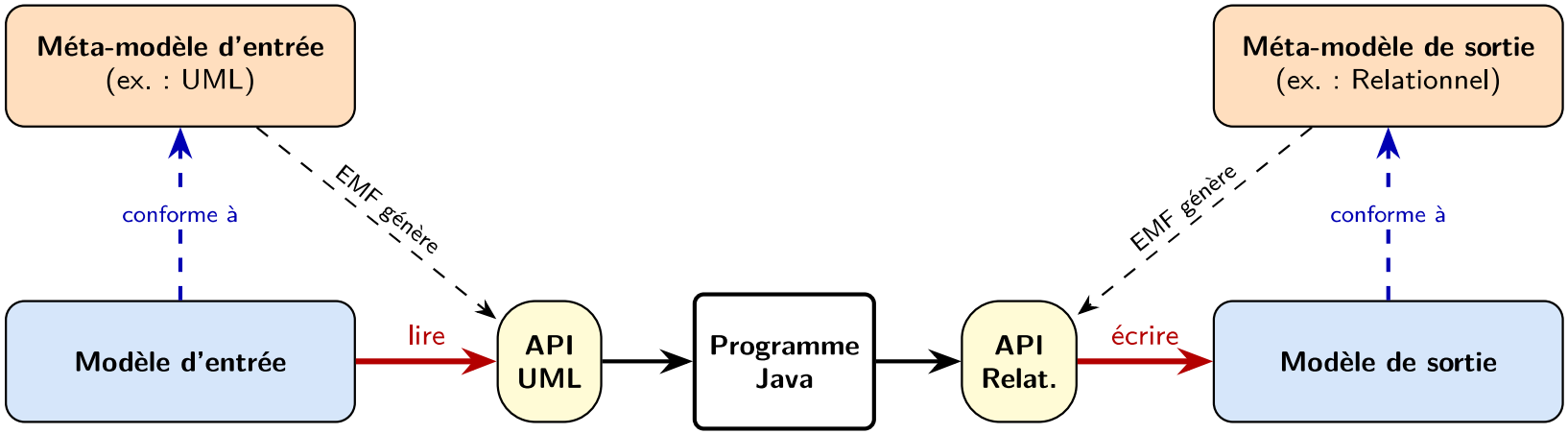
Résultat pour la classe Livre du PIM

```
public class Livre {
    private String isbn;
    private String titre;
    private String auteur;
}
```

- Un fichier .java est créé **par classe** du PIM.
- javaType() est une requête qui traduit String, Integer, Date. . . en types Java : c'est le **PDM**.

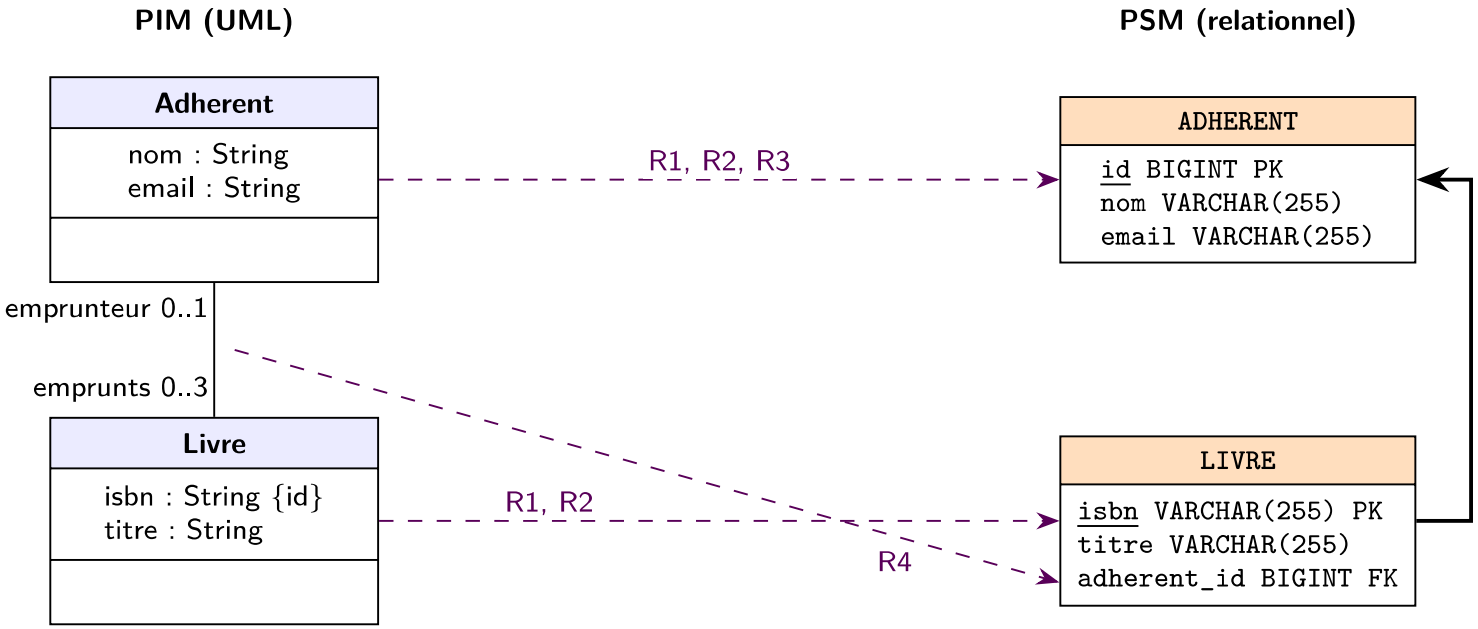
Transformation par programmation

→ Principe



- La transformation est un **programme** (Java) qui **lit** le modèle source et **écrit** le modèle cible.
- Les **API** sont générées automatiquement à partir des méta-modèles (EMF : RelFactory, getClasses()...).
- Avantage : toute la puissance d'un langage. Inconvénient : beaucoup de code, règles moins visibles.

Transformation par programmation



Règle	Source (PIM)	Cible (relationnel)
R1	Classe C	Table nommée C en majuscules
R2	Attribut a de type T	Colonne a de type SQL(T) ; clé si a est {id}
R3	Classe sans attribut {id}	Colonne id BIGINT clé primaire
R4	Association 0..1 (A) — * (B)	Dans B : colonne a_id + clé étrangère vers A

Transformation par programmation

→ Exemple : le programme Java (API EMF) – règles R1 à R3

```java
private Map<Classe, Table> tables = new HashMap<>(); // trace

public Schema transformer(Modele pim) {
  RelFactory f = RelFactory.eINSTANCE;          // API générée par EMF
  Schema s = f.createSchema();
  s.setNom(pim.getNom());
  for (Classe c : pim.getClasses()) {           // R1 : Classe -> Table
    Table t = f.createTable();
    t.setNom(c.getNom().toUpperCase());
    for (Attribut a : c.getAttributs())         // R2 : Attribut -> Colonne
      t.getColonnes().add(colonne(a.getNom(),
            sqlType(a.getType()), a.isEstIdentifiant()));
    if (sansIdentifiant(c))                     // R3 : colonne id
      t.getColonnes().add(0, colonne("id", "BIGINT", true));
    s.getTables().add(t);
    tables.put(c, t);                           // trace Classe -> Table
  }
  for (Association as : pim.getAssociations()) regleR4(as);
  return s;
}
```

- RelFactory, createTable(), getClasses()... sont **générés par EMF** à partir des méta-modèles.
- La Map tables garde la **trace** « quelle classe a donné quelle table » : elle sert pour R4.
- Le programme parcourt le modèle **comme n'importe quel objet Java**.

Transformation par programmation

→ Exemple : le programme Java (API EMF) – règle R4 et PDM

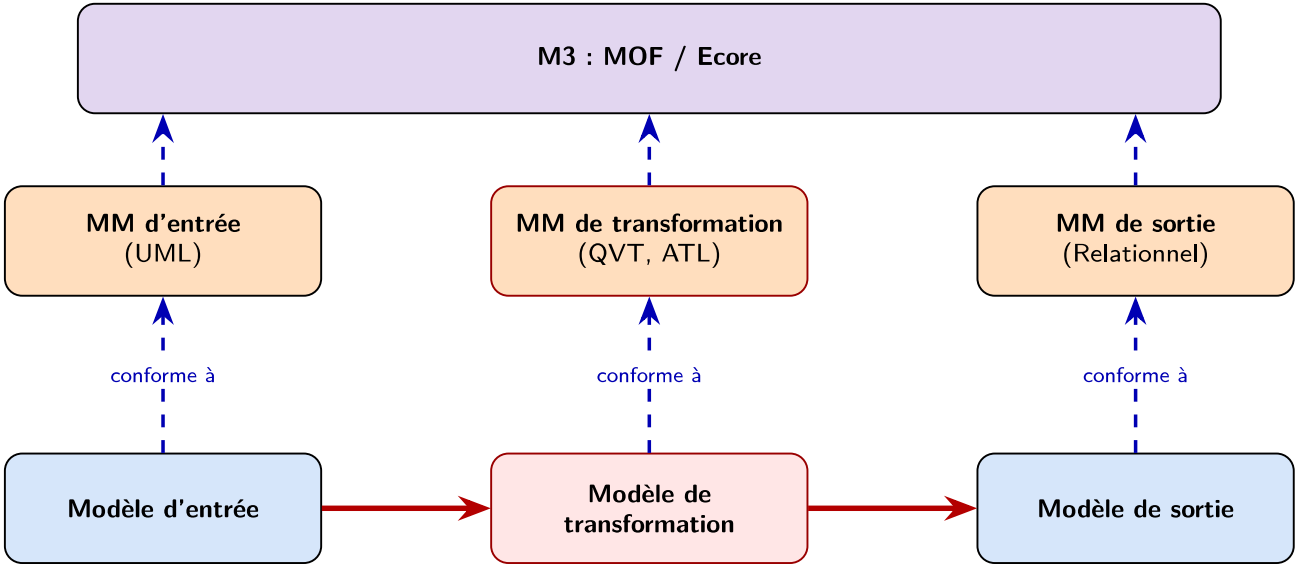
```
void regleR4(Association as) {                  // R4 : association -> FK
  Table un = tables.get(as.getDe());             // côté 0..1
  Table plusieurs = tables.get(as.getVers()); // côté *
  Colonne fk = colonne(un.getNom().toLowerCase() + "_id", "BIGINT", false);
  plusieurs.getColonnes().add(fk);
  CleEtrangere ce = RelFactory.eINSTANCE.createCleEtrangere();
  ce.setColonne(fk);
  ce.setReference(un);
  plusieurs.getCleEtrangeres().add(ce);
}

String sqlType(TypePrimitif t) {                // le PDM SQL
  switch (t) {
    case STRING:  return "VARCHAR(255)";
    case INTEGER: return "INTEGER";
    case BOOLEAN: return "BOOLEAN";
    default:      return "DATE";
  }
}
```

- R4 utilise la **trace** créée par R1 (tables.get(...)).
- sqlType() code le **PDM** SQL : la correspondance des types.
- Appliqué au PIM bibliothèque, on obtient exactement le PSM relationnel vu plus haut (ADHERENT, LIVRE + adherent_id).

Transformation par modélisation

→ Principe



*la transformation est elle-même un **modèle** : on peut la vérifier, la transformer, la réutiliser*

- La transformation est un **modèle** conforme au méta-modèle d'un langage de transformation (QVT de l'OMG, ATL d'Eclipse).
- Les règles sont **déclaratives** : « tout X devient un Y », le moteur s'occupe du parcours et des liens.

Transformation par modélisation

→ Exemple : la même transformation en ATL

```
module PIM2Relationnel;
create OUT : Rel from IN : Pim;

helper def : sqlType(t : Pim!TypePrimitif) : String =
  if t = #STRING then 'VARCHAR(255)'
  else if t = #INTEGER then 'INTEGER'
  else if t = #BOOLEAN then 'BOOLEAN' else 'DATE' endif endif endif;

rule Classe2Table {                          -- R1
  from c : Pim!Classe
  to   t : Rel!Table ( nom <- c.nom.toUpper(),
                       colonnes <- c.attributs )
}
rule Attribut2Colonne {                      -- R2
  from a : Pim!Attribut
  to   k : Rel!Colonne ( nom <- a.nom,
                         type <- thisModule.sqlType(a.type),
                         estCle <- a.estIdentifiant )
}
```

- **from** : l'élément source filtré.
- **to** : l'élément cible créé.
- <- : affectation d'une propriété.
- colonnes <- c.attributs : ATL utilise **automatiquement** la règle Attribut2Colonne (liens de traçabilité).
- **helper** : fonction réutilisable (ici, le PDM SQL).

! ~15 lignes ATL contre ~25 lignes Java pour R1 et R2 : les règles sont **lisibles** et **réutilisables**.

Approches de transformation

→ Comparaison

	Par template	**Par programmation**	**Par modélisation**
La transformation est...	un texte à trous	un programme	un modèle
Cible typique	texte (M2T)	modèle ou texte	modèle (M2M)
Outils	XSLT, Acceleo	Java + EMF	ATL, QVT
Points forts	simple, lisible	puissant, flexible	déclaratif, réutilisable, traçable
Points faibles	logique complexe difficile	verbeux, règles cachées	langage à apprendre
Dans la bibliothèque	PSM → Adherent.java	PIM → schéma relationnel	PIM → schéma relationnel

Exercices et synthèse

Savez-vous classer, projeter et transformer ?

Exercices

→ Exercice 1 : transformation par template (énoncé)

Transformer ce document XML en page HTML (tableau)

```
<test>
  <jeu id="1"><nom>Guild Wars</nom>
     <type>Jeu de rôle en ligne</type><prix>Environ 20 €</prix
      ></jeu>
  <jeu id="2"><nom>Super Mario Galaxy</nom>
     <type>Jeu de plate-forme</type><prix>30-50 €</prix></jeu>
  <jeu id="3"><nom>Mario Sokoban</nom>
     <type>Jeu de réflexion</type><prix>Gratuit</prix></jeu>
</test>
```

Résultat attendu

N°	Nom	Type	Prix
1	Guild Wars	Jeu de rôle en ligne	Environ 20
2	Super Mario Galaxy	Jeu de plate-forme	30-50 €
3	Mario Sokoban	Jeu de réflexion	Gratuit

Indice : une ligne <tr> par élément jeu ; @id lit un **attribut**.

Exercices

→ Exercice 1 : correction

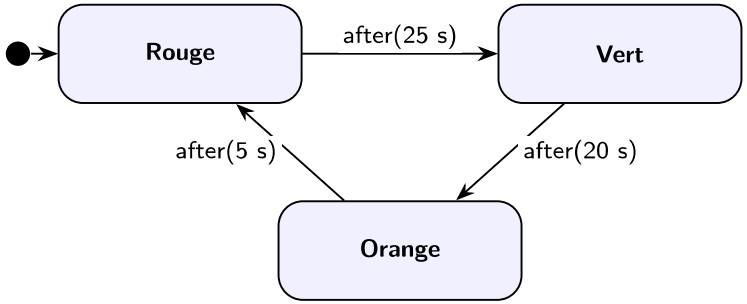
```
<xsl:template match="/">
 <html><body>
  <table border="1">
   <tr><th>N°</th><th>Nom</th><th>Type</th><th>Prix</th></tr>
   <xsl:for-each select="/test/jeu">
    <tr>
     <td><xsl:value-of select="@id"/></td>
     <td><xsl:value-of select="nom"/></td>
     <td><xsl:value-of select="type"/></td>
     <td><xsl:value-of select="prix"/></td>
    </tr>
   </xsl:for-each>
  </table>
 </body></html>
</xsl:template>
```

- for-each select=”/test/jeu” : **une ligne** par jeu.
- value-of select=”@id” : l’**attribut** id.
- value-of select=”nom” : l’**élément** fils nom.
- Ajouter < ?xml-stylesheet type=”text/xsl” href=”jeux.xsl” ?> dans jeux.xml pour tester dans un navigateur.

Exercices

→ Exercice 2 : transformation par programmation (énoncé)

Le modèle : la machine à états du feu



Écrire une méthode Java versXml(StateMachine sm) qui parcourt le modèle (API EMF du méta-modèle états-transitions) et produit le document XML ci-contre.

Résultat attendu

```
<stateMachine name="FeuTricolore">
  <state name="Rouge" initial="true"/>
  <state name="Vert"/>
  <state name="Orange"/>
  <transition from="Rouge"  to="Vert"   event="after(25 s)"/>
  <transition from="Vert"   to="Orange" event="after(20 s)"/>
  <transition from="Orange" to="Rouge"  event="after(5 s)"/>
</stateMachine>
```

Exercices

→ Exercice 2 : correction

```
public static String versXml(StateMachine sm) {
  StringBuilder sb = new StringBuilder();
  sb.append("<stateMachine name=\"" + sm.getName() + "\">\n");
  for (State s : sm.getStates())                  // une balise par état
    sb.append("  <state name=\"" + s.getName() + "\""
        + (s == sm.getInitial() ? " initial=\"true\"" : "") + "/>\n");
  for (Transition t : sm.getTransitions())        // une balise par transition
    sb.append("  <transition from=\"" + t.getSource().getName()
        + "\" to=\"" + t.getTarget().getName()
        + "\" event=\"" + t.getEvent() + "\"/>\n");
  return sb.append("</stateMachine>\n").toString();
}
```

- C'est une transformation **M2T par programmation** : le texte est construit « à la main ».
- Question : comment écrire la même chose **par template** ? (réponse : avec Acceleo, voir le TP).

Question flash

?

On dispose d'une ancienne application PHP + MySQL de gestion de bibliothèque. On veut une nouvelle version Java/JPA. Proposer la suite de transformations MDA et donner, pour chacune, son type (sens, endogène/exogène, M2M/M2T).

Question flash

?

On dispose d'une ancienne application PHP + MySQL de gestion de bibliothèque. On veut une nouvelle version Java/JPA. Proposer la suite de transformations MDA et donner, pour chacune, son type (sens, endogène/exogène, M2M/M2T).

Réponse

(1) Code/schéma MySQL → PSM relationnel : **T2M**, rétro-ingénierie. (2) PSM → **PIM** : rétro-ingénierie, exogène, verticale, M2M. (3) PIM → PIM : **raffinement** éventuel (endogène). (4) PIM → PSM Java/JPA : **projection**, exogène, verticale, M2M. (5) PSM → code Java : **génération**, M2T.

À retenir

CIM

Le **besoin** métier, sans informatique (cas d'utilisation, exigences).

PIM

La **solution** sans technologie (classes + OCL). Le cœur de MDA.

PDM

La **plateforme** et les règles pour s'y projeter.

PSM

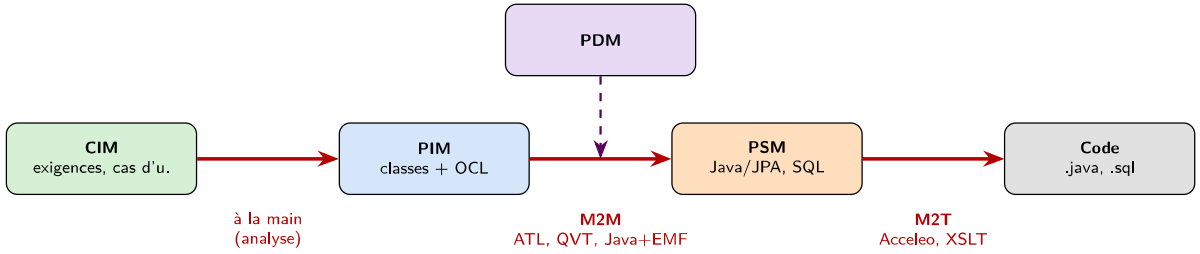
Le PIM **projeté** sur une plateforme (Java/JPA, SQL...).

Transformations

Règles écrites sur les **méta-modèles** : M2M (ATL, QVT, Java), M2T (Acceleo, XSLT).

Le gain

Métier écrit **une fois**, **réutilisé** sur chaque plateforme ; code généré, cohérent, traçable.



Glossaire

Terme	Définition courte
MDA	Model Driven Architecture : démarche IDM de l'OMG (2001)
CIM / PIM / PSM	modèle indépendant de l'informatique / de la plateforme / spécifique à une plateforme
PDM	Platform Description Model : description d'une plateforme et des règles de projection
Raffinement / projection	PIM→PIM plus précis / PIM→PSM pour une plateforme
Rétro-ingénierie	transformation vers un niveau plus abstrait (Code→PSM→PIM)
Endogène / exogène	même méta-modèle / méta-modèles différents
M2M / M2T / T2M	modèle→modèle / modèle→texte / texte→modèle
XSLT, Acceleo / ATL, QVT	langages de transformation par template / par modélisation

Prochaines séances

TD : du CIM au code sur des études de cas (bibliothèque, feu, jeux XML). **TP** : XSLT, transformation Java + EMF, ATL et Acceleo sur la bibliothèque.